AVEVA

CONTINUAL PROGRESSION

AVEVA Plant (12 Series)

Programmable Macro Language: Macros and Functions

TM-1401

TRAINING GUIDE

www.aveva.com

Revision Log

Date	Revision	Description of Revision	Author	Reviewed	Approved
22/01/2009	0.1	Issued for Review	BT		
26/01/2009	0.2	Reviewed	BT	EJW	
29/01/2009	1.0	Approved for Training 12.0.SP3	BT	EJW	RP
15/01/2010	1.1	PML Training Manuals Split. Issued for Review	JW		
27/01/2010	1.2	Reviewed	JW	EJW	
20/05/2010	2.0	Issued for Training 12.0.SP5	JW	EJW	RP

Updates

All headings containing updated or new material will be highlighted.

Suggestion / Problems

If you have a suggestion about this manual or the system to which it refers please report it to the AVEVA Group Solutions Centre at **gsc@aveva.com**

This manual provides documentation relating to products to which you may not have access or which may not be licensed to you. For further information on which products are licensed to you please refer to your licence conditions.

Visit our website at **http://www.aveva.com**

Disclaimer

Trademarks

Copyright

Printed by AVEVA Solutions on 26 May 2010

AVEVA Solutions Ltd, High Cross, Madingley Road, Cambridge, CB3 0HB, United Kingdom.

Contents

CHAPTER 1

1 Introduction

This manual is designed to give an introduction to the AVEVA Plant Programming Macro Language. There is no intention to teach software programming but only provide instruction on how to customise PDMS using Programmable Macro Language (PML) in AVEVA Plant.

This training guide is supported by the reference manuals available within the products installation folder. References will be made to these manuals throughout the guide.

1.1 Aim

The following points need to be understood by the trainees:

- Understand how PML can be used to customise AVEVA Plant
- Understand how to create Macros and Functions
- Understand how to use the built-in features of AVEVA Plant

1.2 Objectives

At the end of this training, you will have:

- A broad overview of Programmable Macro Language (PML)
- Knowledge of basic coding practices and conventions
- An understanding of how PML can interact with the Design model

1.3 Prerequisites

The participants must have completed an AVEVA Basic Design Course and have a familiarity with PDMS. Previous experience of programming is of benefit – but not necessary

1.4 Course Structure

Training will consist of oral and visual presentations, demonstrations, worked examples and set exercises. Each trainee will be provided with some example files to support this guide. Each workstation will have a training project, populated with model objects. This will be used by the trainees to practice their methods, and complete the set exercises.

1.5 Using this Guide

Certain text styles are used to indicate special situations throughout this document, here is a summary;

Menu pull downs and button press actions are indicated by **bold dark turquoise text**.

Information the user has to Key-in **'will be red and BOLD'**

Additional information will be highlighted

Reference to other documentation will be separate

System prompts should be bold and italic in inverted commas i.e. ***'Choose function'***

Example files or inputs will be in the `courier new font` with colours and styles used as before.

2 PML Overview

Programmable Macro Language (PML) is the customisation language used by AVEVA Plant and it provides a mechanism for users to add their own functionality to the AVEVA Plant software family. This functionality could be as simple as a re-naming macro, or as complex as a complete user-defined application (and everything in between).

PML is a coding language specific to AVEVA products based on the command syntax that is used to drive PDMS. As the products are developed, PML is also improved providing new functionality and bringing it closer to other object-orientated programming languages, while still retaining the command syntax. Although it is one language, there are three distinct parts to it:

- PML 1 the first version of PML based on command syntax. String based, but provides IF statement, loops, variables, error handling
- PML 2 object oriented language extending the ability of PML. Use of functions, objects and methods to process information
- PML .NET provides the platform in PML to display and use objects created in other .NET languages

2.1 PML 1 – String Based Command Syntax

When PML is written as command syntax, PDMS processes it as individual command lines and runs them in sequence - as if they had been typed directly into the command window.

A simple macro is likely to be written completely in command syntax and allows users to re-run popular commands. Saved as an ASCII file, the macro can be run in PDMS through the command window (by typing `$m/FILENAME` or by dragging and dropping).

2.1.1 Example of A Simple Command Syntax Macro

The following is an example of a simple macro that can be used to create an Equipment element. As each line is run in order, the same piece of equipment will be created each time the macro is run.

```
NEW EQUIP /ABCD
NEW BOX
XLEN 300 YLEN 400 ZLEN 600
NEW CYL DIA 400 HEI 600
CONN P1 TO P2 OF PREV
```

Macros can be extended to include IF statements, DO loops, variables and error handling (these are explained further in Sections 3.10 and 3.11) If additional information is typed onto the same line as the call for the macro, then these become input parameters and are available for use.

i.e. `$M/BUILDBOX 100 200 300`

This means that the extra 3 values after the macro name are treated as parameters 1, 2 and 3. When entered in this way, these parameters are then available for use in the macro instead of fixed values.

2.1.2 Examples of Command Syntax

The following are examples of command syntax that can be typed directly into the command window (or used within a macro):

Working with Elements and attributes:

- To find out attribute information about the current element, type `Q ATT`
- To create an new element (for example, BOX), type `NEW BOX`
- To find out a specific attribute (for example NAME), type `Q NAME`

- To set a value to an attribute on the current element (for example XLEN), type **XLEN 300**

Manipulating the drawlist in DESIGN:

- To add the current element to the drawlist, type **ADD CE**
- To add a specific element below a piece of equipment, type **ADD ONLY /E1301-S1**
- To remove all the elements from the drawlist, type **REM ALL**

Annotating elements in DESIGN:

- To label the current element with its name, type **MARK CE**
- To label the element /PIPE with its name, type **MARK /100-B-1-B1**
- To remove the label from the current element, type **UNMARK CE**
- To remove all labels, type **UNMARK ALL**
- To mark all branch elements with their names, type **MARK WITH (NAME) ALL BRAN**
- To mark all large valve elements with their specification reference, type **MARK WITH (NAME OF SPREF) ALL VALV WHERE CPAR[1] GT 100**

Using Aid graphics:

- To draw an unnumbered graphic aid line, type **AID LINE E0N0U0 TO E0N1000U1000**
- To draw a sphere aid (aid number 1000), type **AID SPHERE NUM 1000 E0N0U0 DIAM 200**
- To remove all unnumbered graphical aid lines, type **AID CLEAR LINE UNN**
- To remove all number 1000 sphere aids, type **AID CLEAR SPHERE 1000**
- To remove all graphical aids, type **AID CLEAR ALL**

Adding colour to the 3D model:

- To apply the standard enhance colour to the current element, type **ENHANCE CE**
- To apply colour 10 to element /PIPE, type **ENHANCE /PIPE COL 10**
- To remove all colour from the 3D model, type **UNENHANCE ALL**

Position and orientation of elements:

- To move the current element 1000mm in a N45E direction, type **MOVE N45E DIST 1000**
- To move the current element E, until it is inline with /BOX, type **MOVE E THRU /BOX**
- To rotate the current element around it's origin (pointing up) by 45, type **ROTATE BY 45 ABOUT U**
- To relatively move the current element east by 1000mm, type **BY E 1000**
- To explicitly position the current element, type **AT E0 N1000 U2000**
- To position & orientate the current element based on /EQUIP-N1, type **CONNECT P2 TO P0 OF /EQUIP-N1**
- To position & orientate the current element based on the previous, type **CONNECT P3 TO P1 OF PREV**

2.1.3 Syntax Graphs

Many examples of command syntax are provided in the various reference manuals within the installation folder of AVEVA Plant. For each a syntax graph is provided to show the various combinations of syntax available:

```
>-- ADD --+-- Only --+    .----<-------.
          |          |   /             |
          `----------*-- <selatt>   ---+-- COLour <colno> -->
                                       |
                                       `-->
```

The above syntax graph is for the **ADD** syntax (used to add elements to the drawlist). From the graph it can be seen that only two words are required (e.g **ADD /PIPE**) but others can be included (e.g. **ADD ONLY /PIPE1 /PIPE2 COL 3**).

More examples of command syntax and the supporting syntax graphs can be found in the relevant reference manuals provided with AVEVA Plant.

2.2 PML 2 – Object-Orientated Programming

PML 2 is almost an object oriented language and provides most features of other Object-Orientated Programming (OOP) languages. Operators and methods are polymorphic and overloading of methods is supported. There is no inheritance and there is no concept of public or private variables.

PML 2 provides for classes of built-in, system-defined and user-defined object types. Objects have members (their own variables) and methods (their own functions). All PML variables are an instance of a built-in, system-defined or user-defined object type.

Through the use of method concatenation, it is possible to achieve multiple operations in a single line of code. This means that PML 2 methods are typically shorter and easier to read than the PML 1 equivalent. While most PML 1 macros will still run within PDMS, PML 2 brings many new features that were previously unavailable. If used together with command syntax it must be expressed as a string before use.

2.2.1 Features of PML 2

The main features of PML 2 are:

- Available Variable Types - STRING, REAL, BOOLEAN, ARRAY
- Built in Methods for commonly used actions
- Global Functions supersede old style macros
- User Defined Object Types
- PML Search Path (PMLLIB)
- Dynamic Loading of Forms, Functions and Objects

2.2.2 Examples of Object-Orientated PML

Declaration of objects (variables):

- To declare a local variable as a REAL 3, type **`!realVariable = 3`**
- To declare an global BOOLEAN variable as false, type **`!!booleanVariable = FALSE`**
- To set the third value in an ARRAY as Fred, type **`!arrayVariable[3] = |Fred|`**
- To declare a variable as an empty position object, type **`!position = object position()`**

Finding out information about objects (variables)

- To find out the value of an object, type **`q var !exampleObject`**
- To find out the type of an object, type **`q var !exampleObject.objectType()`**
- To find out what members an object has, type **`q var !exampleObject.attributes()`**

Methods available on objects:

- To find out what methods are available on an object, type **`q var !exampleObject.methods()`**
- To find out is an object has been given a value, type **`q var !exampleObject.set()`**
- To find out how large an ARRAY variable is, type **`q var !arrayVariable.size()`**
- To find out how long a STRING variable is, type **`q var !stringVariable.length()`**

Clearing an object

- To empty an object, type **`!exampleObject.clear()`**
- To delete an object that is no longer needed, type **`!exampleObject.delete()`**

2.2.3 Software Customisation Reference Manual

Many examples of the major objects available in AVEVA Plant are provided in the Software Customisation Reference Manual with the products installation folder. For each object, the **members** and **methods** are listed and explained.

- For each of the members the name, type and purpose is provided
- For each method the name, argument types, returned object type and purpose are provided.

Refer to the Software Customisation Reference Manual for more information about the major objects.

2.3 PML Objects

Every variable defined with PML has an object type. This type is set when the variable is defined and is fixed for the life of the variable. When assigning a value to a variable, the object type needs to be considered, otherwise an error may occur. There are three groups of object types available:

- Built-in (e.g. String, Real, Boolean and Array)
- System-defined (e.g. Position, Orientation)
- User-defined

Refer to the Software Customisation Reference Manual for more examples of System-defined objects

When a variable is declared as a specific object type, it is given all the **members** (attributes) and **methods** of the object definition. This means standard groupings can be setup and that code repetition can be avoided.

A user-defined object provides an opportunity to group object types together for a specific purpose. Once grouped as an object, it can be assigned to a variable and used as any other object. The following are examples of two user-defined objects:

```
define object FACTORY
  member .name is STRING
  member .workers is REAL
  member .output is REAL
endobject

define object PRODUCT
  member .productCode is STRING
  member .total is REAL
  member .site is FACTORY
endobject
```

These objects would be defined as two separate object files (.pmlobj) and loaded into PDMS. You will notice that the object **PRODUCT** is able to have a member which is another user-defined object. This means that the **PRODUCT** object has access to all the members and methods of a **FACTORY** object.

2.3.1 Creating Variables (Instances of Objects)

There are two types of variables in PML (1) **Local** and (2) **Global**

The difference between the two is that global variables last for the whole PDMS session and can be referenced directly from other PML routines. Local variables are only available for use within the routine which defined them. The variables a declared with a single **'!'** for local and a double **'!!'** for global

- **!localVariable** – a local variable
- **!!globalVariable** – a global variable

A variable object-type can be implied from the assigned value:

```
!name = |Fred|          $* To create a LOCAL, STRING variable
!!answer = 42           $* To create a GLOBAL, REAL variable
!!flag = TRUE           $* To create a GLOBAL, BOOLEAN variable
```

It is also possible to define a variable as an object-type without an initial value. The value is therefore **UNSET**

```
!name = STRING()        $* To create a LOCAL, UNSET STRING variable
!!answer = REAL()       $* To create a GLOBAL, UNSET REAL variable
!array = BOOLEAN()      $* To create a LOCAL, UNSET ARRAY variable
```

2.3.2 Naming Conventions

It is common practice to follow a naming convention when defining objects and variables. Use upper case for the name of the object type and mixed upper and lower case for the variable.

For example, the type might be **WORKERS** while the name of the variable might be **numberOf Workers.**

ⓘ *Notice that to make the variable name more meanful, full words are used and upper case letters are used to indicate the words that make up the variable.*

Variable names can be 16 alpha-numeric characters long. They should not start with a number or contain any spaces or full stops (full stops are used to indicate methods and members – explained later).

AVEVA uses a CD prefix on most of its global variables. New functionality does not use a prefix so all new PML must be checked for name clashes. Using your own prefix could help avoid this.

2.3.3 Using the Members of an Object

When a variable is declared as an object-type, it is also given all the objects members and methods. For example:

```
!newPlant = object FACTORY()
```

After being declared as above (using the **OBJECT** keyword and ending with a double bracket) the local variable !newPlant is now a **FACTORY** object and has the same members as the **FACTORY** object. These members are available to store information and can be assigned values in the foll
owing way:

```
!newPlant.name = |ProcessA|
!newPlant.workers = 451
!newPlant.output = 2000
```

ⓘ *Notice the use of a dot between the variable and its member. This methods works providing the word after the dot is a valid member of a the variable object-type*

Once assigned a value, this value is available for use. For example:

```
!numberOfWorkers = !newPlant.workers
```

This creates a new real local variable and assigns it the value 451

2.3.4 Special Objects Used in PDMS

In a standard PDMS DESIGN session, there are number of specialised objects which are loaded and used by standard product. These objects should not be deleted or overwritten, but are available for use. The particularly useful objects are:

- !!CE - a global **DBREF** object which tracks and represents the current element
- !!ERROR - a global **ERROR** object which holds information about the last error
- !!PML - used to obtain file path strings through the **.getPathName()** method
- !!ALERT - used to provide popup feedback to users
- !!AIDNUMBERS - used to manage aid graphics
- !!APPDESMAIN - the form which represents the main **DESIGN** interface

2.4 PML Functions and Methods

Functions and methods provide the actions of PML. When called, a function or method will run through the lines of PML it contains in order – just like a PML 1 style macro. Functions and methods may be passed arguments and even return values. A function which does not return a value is typically referred to as a PML Procedure.

A function and a method are written in the same style, the only difference is where the definition is stored and how it is called. A function is a global method (stored in its own file) and can be called directly on the command line (e.g. **call !!exampleFunction()**) while a method is local to the object it is defined within (e.g. **!exampleObject.exampleMethod()**)

Arguments become local variables within the function/method is the object-type needs to be declared within the definition. The returned object-type is also defined. For example:

```
define function !!area( !length is REAL, !width is REAL) is REAL
  !area = !length * !width
  return !area
endfunction
```

In this example, the function **!!area** is expecting two real arguments. The two arguments are expressed as local variables which are multiplied together to calculate the local variable **!area**. Using the return keyword, the variable **!area** is then returned. If you called the function in following way, you would expect the variable **!area** to have a value of 2400:

```
!area = !!area(12, 200)
```

Functions will be explained further with more examples in chapter 4 and methods on objects will be covered further in chapters 5&6

ⓘ *It is now common practice to write all macros as a functions or procedures*

2.4.1 Arguments of Type ANY

When defining an argument within a method or function, you may declare it as an **ANY** object. This means that the argument can be of any object-type. When **ANY** is used, no argument check is carried out meaning that there may be a subsequent error in the code. For this reason, **ANY** should be used with special consideration:

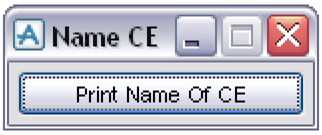
```
define function !!argumentType(!argument is ANY)
  !type = !argument.objecttype()
  return !type
endfunction
```

2.5 PML Forms

For users of PDMS, the concept of a form should already be familiar. In terms of PML, a form is a global variable of the form name (for example **!!exampleForm**). A form is capable of owning members and methods (like any other object) but there are a set of predefined members which can be used to put gadgets on the form (buttons, lists, options etc). Some of these gadgets can be given callbacks, which can activate a standard command or call a function or object method (including the methods defined within the form).

```
setup form !!nameCE
  !this.formTitle = |Name CE|
  button .button |Print Name Of CE| call |!this.print()|
exit

define method .print()
  $p $!!ce.flnn
endmethod
```



The above example is the definition of a form called nameCE. Saved within one file (.pmlfrm), it defines two form members (a predefined member for the title and a new button gadget) and a form method.

The gadgets defined on a form are objects-types as well. This means that when a button is defined, all the members and methods of a button are available.

!this is a special local variable and using it replaces the need to reference the owning object directly. For example, to call the method **.print()** from within the form, type **!this.print()** and to call the method from outside the form, type **!!nameCE.print()**

2.6 PDMSUI Environment Variable

All PML1 Macros are in a directory structure pointed at by the PDMS environment variable **PDMSUI**. The PDMSUI environment variable is set in the .bat file that is used to load PDMS (typically within evars.bat). To set the environment variable, the following line is needed in the .bat file:

```
set PDMSUI=C:\AVEVA\plant\PDMS12.0.3\pdmsui
```

The purpose of an environment variable is to reduce the length of the command used to call a macro. This means that **$M/%PDMSUI%\DES\PIPE\MPIPE** can be typed instead of the full file path.

In standard product, this process is shortened further as all PML1 macros and forms are called using synonyms. For example, the macros associated with piping are called using the synonym **CALLP**:

```
$S CALLP=$M/%PDMSUI%/DES/PIPE/$s1
CALLP MPIPE
```

ⓘ *If all synonyms are killed then PDMS will cease to function as normal*

2.7 PMLLIB Environment Variable

All PML 2 objects and functions are in a directory structure pointed at by the PDMS environment variable **PMLLIB**. As with the PDMSUI variable, this is set in the .bat file which runs PDMS. To set the environment variable, the following line is needed in the .bat file:

```
set PMLLIB= C:\AVEVA\plant\PDMS12.0.3\pmllib
```

The PMLLIB environment variable differs because it can be searched dynamically. This means that the individual files do not need to be referenced directly and can be called by name. This is possible because PDMS compiles a **pml.index** file which sits in the PMLLIB folder and provides the path to all suitable files within it. For example, to load and show a form the command is `show !!exampleForm` (no need to reference the file path at all)

If a new file is created or the PMLLIB variable changed then there is a need to update the pml.index file. This can be done by typing `PML REHASH` onto the command window. If there are multiple paths that need updating, type `PML REHASH ALL`

If a pml object has already been loaded into PDMS, but the file definition has changed then the object needs to be reloaded before the changes can be seen. This can be done by typing either `pml reload form !!exampleForm` or `pml reload object EXAMPLEOBJECT`

Although not necessary, it is good practice to organise the files below the PMLLIB folder. A standard PDMS installation orgainises the files based on application and then on forms, functions, objects. This is normally a good starting point for organising customisation.

2.8 Modifications to the PDMSUI and PMLLIB

As PML is developed, it is normal practice to store it in a parallel file structure outside the standard installation directory. This parallel file structure can then be reference by the .bat files. There are a couple of reasons why this is a good idea:

- It keeps the customisation separate from the standard install, so as subsequent versions are installed there will not be a need to move the customised files.
- If any standard files are modified then the originals are still available if required
- If the customisation fails, the standard installation is available to go back to
- Many local PDMS installations can reference the same customisation from a network address

ⓘ *Any changes to AVEVA standard product may cause PDMS to function inappropriately*

It is possible to get PDMS to look in different places for PML and this is done by setting the environment variables to multiple paths. This allows the standard install to be kept separate from user and company customisation. This is done by updating the variable to include another path:

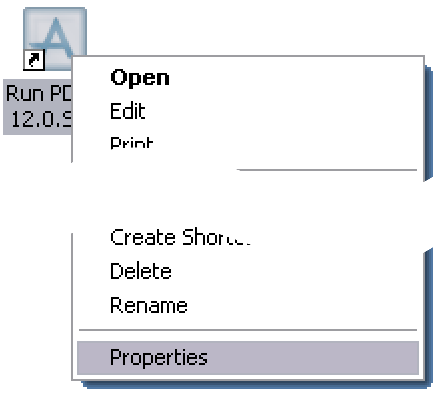
```
Set PDMSUI=c:\temp\pdmsui %pdmsui%
```

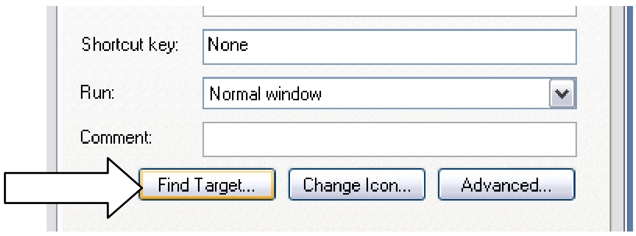
This will put the additional file path in front of the standard (which would have already been defined in the .bat file). This change can also be checked in a PDMS session by typing **`q evar PDMSUI`** or **`q evar PMLLIB`** onto the command window.

Exercise 1 – Updating the Environment Variables

1

- Extract the provided files into the folder **C:\temp**
- Right –click on the icon that opens PDMS. **Choose Properties…** then **Find Target**





2

- Open the file in a suitable text editor
- Add the following lines to pdms.bat (after the line that calls evars.bat)

```
set PDMSUI=c:\temp\pdmsui %pdmsui%
set PMLLIB=c:\temp\pmllib %pmllib%
```

- Save the .bat as a **new file** to the computers desktop. This is now the icon you will use to enter PDMS.

3

- Load PDMS by using the new .bat and enter the Design module

4

- Confirm that the search paths have been correctly updated. Type **`q evar PDMSUI`** or **`q evar PMLLIB`** into the command window.

2.9 General Notes on PML

- PML functions and methods run through their definitions in line order (control logic can be used to alter this order)
- Functions should be written in preference to macros
- Variables are instances of object definitions
- The return command can be used to exit a function/method as well as return a value
- PML files are **ASCII** and can be created/edited in any basic text editor
- PML 1 files are saved under **PDMSUI** folder and PML 2 under the **PMLLIB** folder
- PML 2 objects have specific file extensions (.pmlfnc, .pmlobj and .pmlfrm)
- If new created, PML can be found by typing **`pml rehash all`**
- Once loaded, objects can be reloaded by typing **`pml reload object OBJECT`**
- Once loaded, forms can be reloaded by typing **`pml reload form FORM`**
- When declaring a string, text delimiters must be used. Either **`'single quotes'`** or **`|vertical bars|`**
- File paths of files can be obtained by using **`!!pml.getPathName(|form.pmlfrm|)`**
- The **`$`** is used as an escape charater and can be used to expand a variable before it is read as command syntax. If a **`$`** symbol is actually required, then two must be entered
- To provide comments in statements with PML, one of the following can be used:
  - For a comment line, start the line with two hyphens **`--`**
  - For a comment at the end of a line, start the comment with **`$*`**
  - For a block comment, start with a **`$(`** and end with a **`$)`**
- Variable names are not case sensitive
- String comparisions are case sensitive
- Variable names can be 64 characters long and should not start with a number or contain a dot
- It is good practice to name variables in lower case, using upper case to separate words e.g. !stringLength
- It is possible to abbreviate some command syntax. The required part is shown in upper case with the syntax graphs e.g. **`WIDth`** means that **`WID`** is acceptable when declaring width

3 Macros, Synonyms and Control Logic

A macro is a group of PDMS commands written to a file (in sequence) that can be run together. This removes the need for user to have to enter every line of code separately and that repetitive processes can be run as a macro.

3.1 A Simple Macro

The following example is a macro which creates an EQUI element which owns a BOX and CYLI. To test the example drag and drop the provided file into the command window.

ⓘ *When running this macro in, ensure the Current Element (CE) is a ZONE or below*

```
NEW EQUIP /ABCD
NEW BOX
XLEN 300 YLEN 400 ZLEN 600
NEW CYL DIA 400 HEI 600
CONN P1 TO P2 OF PREV
```

You have created your first macro. To run the macro into PDMS, either drag and drop the file into the command window or type **$M/** followed by the full file path of the saved macro.

e.g. `$M/%PDMSUI%\examples\simpleMac.mac`

3.2 Finding Examples of Command Syntax

While developing a macro there are four main techniques of deriving the command syntax needed:

- DB listing utility — create the required elements in PDMS using the standard appware and use the DB Listing utility to output the information (**Utilities>DB Listing…**)
- $Q syntax — if part of a command is known, the syntax which follows it can be found by typing `$Q` after the command e.g `NEW $Q`
- Using standard product — standard PDMS is supplied with numerous PML files which can be searched for keywords and used for inspiration
- Reference Manuals — the reference manuals supplied with the product focus on command syntax and supply syntax graphs for each case.

As macros develop in complexity and use, it is likely that a mixture of the above will be used.

3.3 Communicating with AVEVA Products in PML

All commands need to be supplied to the command processor as **STRING**s. This is important when working with variables as they may have another object-type. If a **$** symbol is put infront of a variable, PDMS will expand the contents of the variable as a string and then read the line. For example:

```
!componentType = |BOX|
!xLength = 5600
NEW $!componentType XLEN $!xLength
```

This is the equivalent of writing `NEW BOX XLEN 5600`

3.4 Parameterised Macros

Macros can be parameterised. This means instead of hard coding the values in the macro, arguments can be passed to it and used instead. This allows the values to be varied, making the macro flexible. As an example, simpleMac.mac can be parameterised in the following way:

```
NEW EQUIP /$1
NEW BOX
XLEN $2 YLEN $3 ZLEN $4
NEW CYL DIA $3 HEI $4
CONN P1 TO P2 OF PREV
```

To test the example, find the provided example. To run this Macro, parameters need to be passed to it. The four parameters are defined by including the values of the parameters are the macro call:

e.g. `$M/%PDMSUI%\examples\parameterMac.mac ABCDE 300 400 600`

If this macro is dragged and dropped into the command window, the parameters would be undefined and the macro will error. To avoid this, default parameter values can be set at the top of the macro using the **$d=** synatx. For example:

```
$d1=ABCDEF
$d2=300
$d3=400
$d4=600
```

ⓘ *The defined default values will only be used if no parameters are passed to the macro.*

Macros may have up to 9 parameters separated by space. In the below example, ABC, DEF & GHK are seen as separate strings and therefore different parameters:

e.g. `$M/%PDMSUI%\examples\nineParameter.mac ABC DEF GHK 55 66 77 88 99 00`

If a text string is required as a single parameter, it can be entered by placing a **$<** before and a **$>** after the string. **$< $>** act as delimiters and anything in between is interpreted as a single parameter.

e.g. `$M/%PDMSUI%\examples\sevenParameter.mac $<ABC DEF GHK$> 55 66 77 88 99 00`

3.5 Synonyms

Synonyms are abbreviations of longer commands. They are created by assigning a command to a synonym variable. To call the command held by the synonym, just type the name of the synonym

e.g. `$SNewBox=NEW BOX XLEN 100 YLEN 200 ZLEN 300` called by typing `NewBox`

It is also possible to parameterise a synonym:

e.g. `$SNewBox=NEW BOX XLEN $S1 YLEN $S2 ZLEN $S3` called by typing `NewBox 100 200 300`

A synonym can call itself and if it uses the **$/** syntax (return character) it can be used to loop through elements. For example, a new XLEN value can be applied to all elements for level in the hierarchy

e.g. `$SXChange=XLEN 1000 $/ NEXT $/ XChange`

Synonyms can be turned off and on with the `$S-` and `$S+` syntax. To kill a synonym, type `$SXXX=` & to kill all synonyms `$sk`. Standard PDMS relies on synonyms to function. If a required synonym is killed, the product will no longer function properly (requiring a restart)

3.6 Defining Variables

There are number of different methods for defining variables in PDMS:

3.6.1 Numbered Variables

Numbered variables are set by typing a number then value after the command VAR. Examples of this are below:

```
var 1 name
var 2 |hello|
var 3 (99)
var 4 (99 * 3 / 6 + 0.5)
var 117 pos in site
var 118 (name of owner of owner)
var 119 'hello ' + 'world ' + 'how are you'
```

The value of variable 1 can be obtained by typing **q var 1** into the command window. The available variable numbers only go to **119** (there is no 120) and they are module dependant. For these reasons, this technique is no longer commonly used and has been included in this training guide for completeness.

3.6.2 PML 1 Style Variables

The following are some examples of setting variables using the PML 1 style syntax:

```
VAR !name NAME              $* Takes the current element's (CE) name attribute
VAR !pos POS IN WORLD       $* Takes CE position attribute relative to the world
VAR !x |NAME|               $* Sets the variable to the text string |NAME|
VAR !temp (23 * 1.8 + 32)   $* Calculate a value using the expression
VAR !list COLL ALL ELBO FOR CE $* Makes a string array of database references
```

This style of defining variables is still valid and is useful for command syntax that returns a value. It has been included within this guide primarily for information for when old code is upgraded.

3.6.3 PML 2 Style Variables

The following are some examples of setting variables using the PML 2 style syntax:

```
!name = !!ce.name           $* Takes the current element's (CE) name attribute
!pos = !!ce.pos.wrt(/*)     $* Takes CE position attribute relative to the world
!x = |NAME|                 $* Sets the variable to the text string |NAME|
!temp = 23 * 1.8 + 32       $* Calculate a value using the expression
```

These examples show that there are equivalents to the PML 1 style, but because PML 2 is object-based this can be incorporated into the declaration of the variable. Objects will be explained further with examples in chapter 6. There is no single line PML 2 equivalent to the PML 1 COLL syntax. This will be discussed in chapter 7

3.7 Expressions

Expressions are calculations using PML variables. This can be done in a PML 1 or PML 2 style:

```
VAR !z ( |$!x| + |$!y| )        PML 1
!z = !x + !y                     PML 2
```

In the PML1 example, !z is set as a STRING variable. In the PML2 example, !z is returned as a REAL, if !x and !y are REAL

3.7.1 Expression Operators

There are a number of expression operators which are available for use within PML. The numeric operators and functions are the same for both PML 1 and PML 2 styles. For comparison and logic operators there are new PML 2 methods available:

Numeric operators: + - / *
Numeric functions: SIN COS TAN SQR POW NEGATE ASIN ACOS ATAN LOG ALOG ABS INT NINT

PML 1 style Comparison operators: LT GT EQ NE LE GE
PML 1 style Logic operators: NOT AND OR

PML 2 style Comparison methods: .lt() .gt() .eq() .neq() .leq() .geq()
PML 2 style Logic methods: .not() .and() .or()

The PML 2 comparison methods are available on any object-type variable, providing it is compared to a variable of the same type. The PML 2 logic methods are available on the **BOOLEAN** object

For further examples, refer to PDMS Software Customisation Reference Manual

Some examples of expressions in use:

```
!s = 30 * sin(45)
!t = pow(20,2)                          (raise 20 to the power 2 (=400))
!f = (match(name of owner,|LPX|)gt 0)
```

3.7.2 Operator Precedence

When PDMS reads an expression, there is a precedence that applied to it. This should be considered when writing expressions. The order is as follows:

```
()                       e.g.   60 * 2 / 3 + 5        = 45
* /                             60 * (2 / ( 3 + 5))   = 15
+ -
EQ NE GT LT GE LE
NOT
AND
OR
```

3.7.3 PML 2 Expressions

PML 2 expressions may be of any complexity and may derive the required values from PML Functions and Methods and include Form gadget values, object members and methods. For example:

```
!newValue = !!myFunc(!OldVal) * !!form.gadget.val / !myArray.method()
```

3.8 Arrays

An **ARRAY** variable can contain many values, each of which is called an **ARRAY ELEMENT**.

An array is created by defining one of its array elements or it can be initialised as an empty array (i.e. **!x = ARRAY()**). If an **ARRAY ELEMENT** is itself an **ARRAY**, this will create a **Multi-dimensional ARRAY**. For example, type out the following onto the command window to define an array:

```
!x[1] = |ABCD|
!x[2] = |DEFG|
!y[1] = |1234|
!y[2] = |5678|
!z[1] = !x
!z[2] = !y
```

To query the information about !z, type `q var !z`. This will return the following information:

```
<ARRAY>
   [1]  <ARRAY> 2 Elements
   [2]  <ARRAY> 2 Elements
```

To find out more information about the elements within the Multi-dimensional array, type `q var !z[1]` or `q var !z[2][1]`

3.9 Concatenation Operator

Values to be concatenated are automatically converted to **STRING** by the '**&**' operator. Type the following onto the command window and compare this against the results of typing `!d = !a + !b`

```
!a = 64
!b = 32
!m = |mm|
!c = !a & !b & !m
q var !c
```

3.10 DO Loop

A **DO** loop is a way of repeating PML macro, allowing pieces of code to be run more than once. This is useful as it allows code to be reused and reduces the overall number of lines. As an example, try the provided file **%PDMSUI%\examples\doLoop.mac**:

```
DO !loopCounter TO 10
  !value = !loopCounter * 2
  q var !loopCounter !value
ENDDO
```

In the above example, as the loop runs, values of !loopCounter and !value with be printed to the command line for the full range of the defined loop. The step of the loop can be altered by adding **FROM** and **BY** to the loop definition. For example:

```
DO !loopCounter FROM 5 TO 10 BY 2
  !value = !loopCounter * 2
  q var !loopCounter !value
ENDDO
```

3.10.1 DO Loops with BREAK

If you need a loop to run until a certain condition is reached, a **BREAK** command will exit the current loop. This can be used in conjunction with an infinite loop or with a loop with a range. As an example, try the provided file **%PDMSUI%\examples\doBreak.mac**:

```
do !n
  !value = POW(!n, 2)
  q var !value
  BREAK IF (!value GT 1000)
enddo
```

The loop in the example will run until the **BREAK** condition is met. If the condition is never reached, then the code will run **indefinitely**! A DO loop of 1 to 100000 could be used instead as it has an end and won't require PDMS to be crashed to exit the macro

The **BREAK** command can also be called from within a normal **IF** construct. This is typically done if multiple break conditions need to be considered. For example:

```
if (!value GT 1000) then
  BREAK
endif
```

3.10.2 DO Loops with SKIP

It is possible to skip part of the DO loop using the **SKIP** command. This could be useful if parts of a number sequence needs to be missed. As an example, try the provided file **%PDMSUI%\examples\doSkip.mac**:

```
do !n from 1 TO 25
  SKIP IF (!n LE 5) OR (!n GT 15)
  q var !n
enddo
```

The SKIP command can also be called within a normal IF construct (as the **BREAK** command)

3.10.3 DO INDEX and DO VALUES

DO INDEX and **DO VALUES** are ways of looping through arrays. This is an effective method for controlling the values used for the loops. Typically values are collected into an ARRAY variable then looped through using the following:

```
do !X values !ARRAY      $* !X takes each ARRAY element

do !X index !ARRAY       $* !X takes a number from 1 to !ARRAY size
```

Try the provided file **%PDMSUI\examples\doArray.mac**. The **COLLECT** syntax is discussed in chapter 7

```
VAR !zones COLL ALL ZONES FOR SITE
VAR !names EVAL NAME FOR ALL FROM !zones
q var !names

do !x values !names
  q var !x
ENDDO

do !x index !names
  q var !Names[!x]
enddo
```

3.11 IF Statements

An IF statement is a construct for the conditional execution of commands. The commands within the statement will only be run if the conditions are met. In the following example, the code within the IF construct is only run if the expression is **TRUE** (i.e. !number is real and less than 0):

```
if ( !number LT 0 ) then
  !negative = TRUE
endif
```

The expression can be written in any form, providing the answer is BOOLEAN. For example, the following is the same as above but written in a PML 2 style:

```
if ( !number.lt(0) ) then
```

If the value itself is already **BOOLEAN**, a comparison does not need to be made. For example:

```
!booleanVariable = TRUE
if ( !booleanVariable ) then
```

Or `if ( !!exampleFunction() ) then` where !!exampleFunction() returns a **BOOLEAN** result

3.11.1 IF, ELSEIF and ELSE Statements

An IF construct can be extended by adding additional conditions. This is done by adding some **ELSEIF** or **ELSE** statements to it. When an **IF** construct is encountered, PML will evaluate its first condition. If the condition is **FALSE**, PDMS will look to the next **ELSEIF** condition.

Once a condition is found to be TRUE, that part of the code will be run and then the IF construct is complete. If an **ELSE** condition is added, this portion of code will only be run if no other conditions are met. This is a way of ensuring some code runs as part of the construct. As an example, try the provided file **%PDMSUI\examples\numCheck.mac**. Run the macro with one real parameter for the comparison.

```
if ($1 EQ 0) then
  $p Your value is zero
elseif ($1 LT 0) then
  $p Your value is less than zero
else
  $p Your value is greater than zero
endif
```

ⓘ *The ELSEIF and ELSE commands are optional, but there can only be one ELSE command in an IF construct.*

3.12 Branching

PML provides a way of jumping from one part of a macro to another using the **GOLABEL** syntax.

```
LABEL /FRED
...
Some PML code
...
GOLABEL /FRED
```

The next line to be executed after **GOLABEL /FRED** will be the line following **LABEL /FRED**, which could be before or after the **GOLABEL** command. The name of the label can be up to 16 characters (excluding the leading slash)

ⓘ *The use of this method should be limited as it can make code hard to read and therefore debug.*

3.12.1 Conditional Branching

It is possible to add a condition to the **GOLABEL** syntax such that the jump will only be made if the condition is **TRUE**. As an example, type out the following and save it as **c:\temp\pdmsui\conditional.mac**:

```
do !a
$P Processing $!a
  do !b to 10
    !c = !a * !b
    GOLABEL /finished if (!c GT 100)
    $P Product $!c
  enddo
enddo
LABEL /finished
$P Finished with processing = $!a Product = $!c
```

If the expression **!c GT 100** is **TRUE** there will be a jump to label **/finished** and PML execution will continue with the **$P** command. If the expression is **FALSE**, PML execution will continue with the command: **$P Product $!c** and go back through the DO loop.

3.13 Error Handling

An error condition can occur when a command could not complete successfully. This is because of a mistake in the executed macro or function. An error normally has three effects:

- An Alert box appears which the user must acknowledge.
- An error message is outputted to the command line together with a trace back to the error source.
- Any current running PML macros and functions are abandoned.

3.13.1 Error Codes

When an error occurs, an error code is returned to the user along with a message. The following example error is caused when trying to create an EQUI element in the wrong part of the hierarchy.

(41,8) ERROR – Cannot create an EQUI at this level.

Where 41 is the program section which identified the error and 8 is the error code itself.

3.13.2 Error Handling Using the HANDLE Syntax

If the input line which caused the error was part of a PML macro or function, the error may optionally be **HANDLED**. This allows the designer of the macro to limit the errors the user will experience.

As an example, try the provided file **%PDMSUI%\examples\errorTest.mac**. First run the macro at a **SITE** element, then at a **ZONE** element and then again at the same **ZONE**. Compare the return printed lines in the command window.

```
NEW EQUI /ABCD
HANDLE (41, 8)
  $p Need to be at a ZONE or below
ELSEHANDLE (41, 12)
  $p That name has already been used.  Names must be unique
ELSEHANDLE ANY
  $p Another error has occurred
ELSEHANDLE NONE
  $p Everything OK.  EQUI created
ENDHANDLE
```

Notice how the **HANDLE** syntax can differentiate between specific error codes and how it is possible to capture all errors or only run if no error occurs.

3.14 Alert Objects

Alert objects allow user-controlled pop-up forms to be used, providing feedback to the user. The choice on which form type is used and whether one is needed at all is down the the PML designer.

3.14.1 Alert Objects with no Return Value

There are 3 types of alert with no return value:

```
!!alert.error( |You cannot do this!| )
!!alert.message( |Saving your data now| )
!!alert.warning( |Incorrect value!| )
```

By default, all alert forms appear with the relevant button as near to the cursor as possible. To position an alert specifically, X and Y values can be specified as a proportion of the screen size.

```
!!alert.error( |You cannot do this!| , 0.25, 0.1)
```

3.14.2 Alert Objects with Return Value

There are three types of alert which return a value. This value can be subsequently used to as a indication of the decision the user has made (i.e. as part of a IF statement).

3.14.2.1 Confirm Alerts

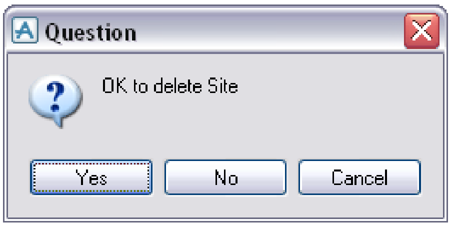
A Confirm alert returns a string of 'YES' or 'NO'

```
!answer = !!alert.confirm( |Are you sure!| )
q var !answer
```

3.14.2.2 Question Alerts

An Answer alert returns a string of 'YES' or 'NO' or 'CANCEL'

```
!answer = !!alert.question( |OK to delete Site| )
q var !answer
```



3.14.3 Input Alerts

An Input alert returns the value entered by the user. If the user does not enter a value, then the default value (set at definition) is returned.

```
!answer = !!alert.input( |Enter
  Width of Floor|,|100| )
q var !answer
```

3.15 Undo and Redo

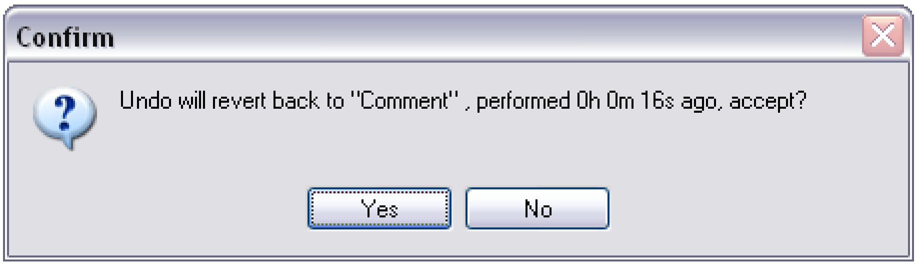
If you build an application that creates or deletes items from the PDMS Database it is good practice to handle the ability to undo the modifications. Undo is a useful feature that users expect to be able to use.

3.15.1 Marking the Database

The undo button goes back to the last database mark or savework so your application should mark the database before modification.

MarkDB 'Comment'

The text string is output if the **Undodb** command is used



3.15.2 Undo and Redo Database Commands

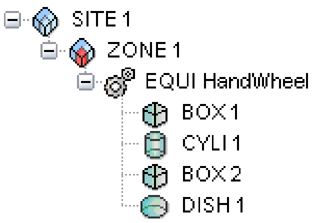
UndoDB	-	Undo the database back to the last database mark
RedoDB	-	Redo the last Undo

These actions can be performed by including the above lines within your code. There is also an UNDOABLE object with built-in methods which can extend this functionality further.

Exercise 2 - The Centre of a Handwheel

1

- Write a macro that will produce the centre of a handwheel (shown to the right)
- It shall be constructed from 4 primitives: 2 boxes, a cylinder and a dish and should create the following hierarchy:



2

- The macro shall create the primitive with the following fixed sizes:

First BOX	X Length = 100; Y Length = 100; Z Length = 100
CYLI	Diameter = 80; Height = 5
Second BOX	X Length = 50; Y Length = 50; Z Length = 15
DISH	Diameter = 50; Height = 15

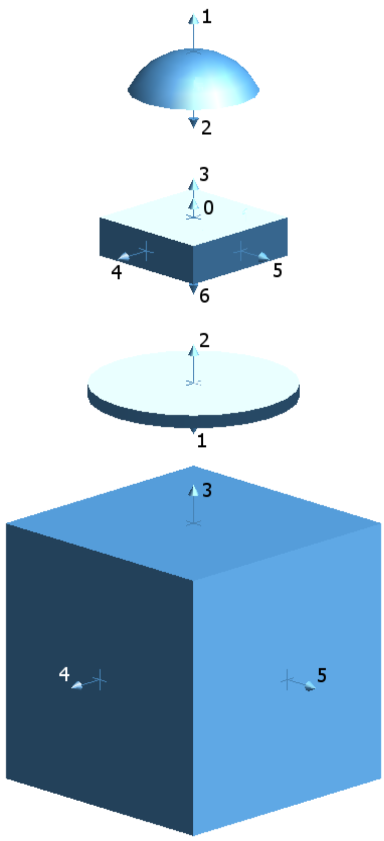
- The primitives should be unnamed, but sit below an EQUI called HandWheel

3

- Let the first BOX be created without specifying a position or orientation.
- Use the CONN syntax to position the next primitive to the first BOX. For example:

```
CONN P1 TO P1 OF PREV
```

- This will connect PPoint 1 (P1) of the current primitive to P1 of the primitive created previously.
- To help with connecting the primitives, refer to the adjacent diagram showing an exploded version of the equipment, with the PPoints identified.



4

- You may wish to add some other syntax to the macro:

  To add the HandWheel to the drawlist
  ```
  ADD /HandWheel
  ```

  To set the limits of the view to the HandWheel
  ```
  AUTO /HandWheel
  ```

  To completely clear the drawlist
  ```
  REM ALL
  ```

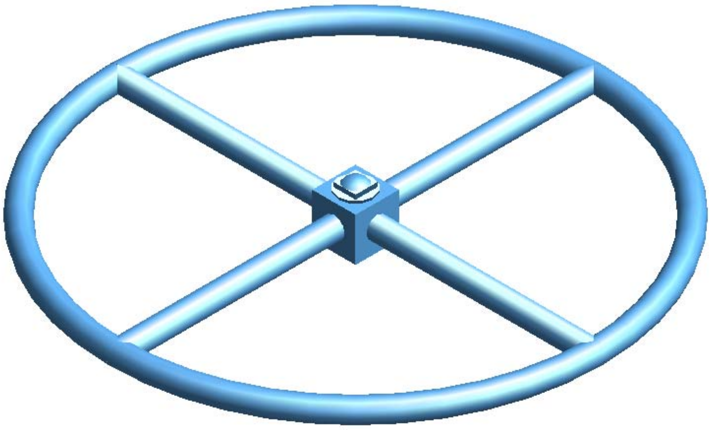
- Save the file as **c:\temp\pdmsui\c1ex2.mac** and run it on the command line by typing `$m/%PDMSUI%\c1ex2.mac` or by dragging and dropping the file into the command window

An example of the completed macro can be found in Appendix B

Exercise 3 - The Full Handwheel

1

- Extend the previous macro to build the remaining parts of the HandWheel (shown below).



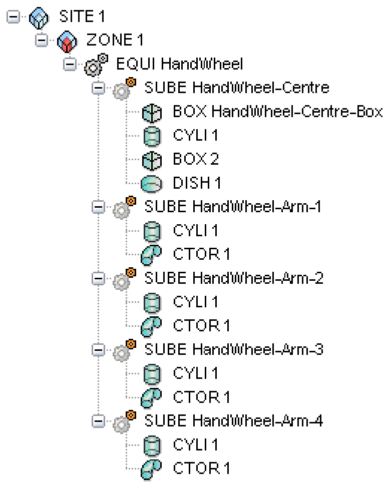
- Information about primitives can be found in Appendix A

2

- It is possible to build the remaining primitives individually, but as there is a rotational centre, a copy-rotate could be used.
- To copy a BOX element (where /XXXX is another BOX. Note: the word PREV is also valid)
  NEW BOX COPY /XXXX
- To rotate the current element (where AAA is the angle and BBB the direction)
  ROTATE BY AAA ABOUT BBB

3

- To help with organization of the primitives of the EQUI, you may wish to add additional SUBE elements. This could help with the copy-rotate methods
- An example hierarchy is shown to the right, but this will depend on your macro



4

- Turn the macro into a parameterized macro. Pass it two parameters (1) the name of the EQUI and (2) the outside diameter of the HandWheel
- Think about how the sizes of the primitives will relate to the outside diameter. You may need to do some calculations before the value can be used
- As parameters are used, set the default values

5

- If any errors occur in your macro, consider error handling or changing the macro so that the error cannot happen
- Save the file as **c:\temp\pdmsui\c1ex3.mac** and run it on the command line by typing **$m/%PDMSUI%\c1ex3.mac** or by dragging and dropping the file into the command window

An example of the completed macro can be found in Appendix B

4 PML Functions

Functions are lines of PML grouped togther in a file designed to do something. When called, the lines of the function are run and the intended action completed.

4.1 Functions Instead of Macros

Functions are designed to replace macros and can contain the same syntax that would have been placed inside a PML 1 style macro. It is intended that any future macros which are created in PML should be written as functions. The following are the reasons why:

- Functions are preloaded through the PMLLIB searchpath
- Functions can accept arguments of specific object-types
- Functions can return values of specific object-types

4.2 Creating a PML Function

A function is saved as a **.pmlfnc** file under the **PMLLIB** search path. The file should be given the same name as the function. For example, if the function is called **!!exampleFunction** then the file should be called **exampleFunction.pmlfnc.**

A function will typically return a value, although arguments are optional. The object-type of the returned information needs to be defined, and is done so at the end of the 'define function' line.

The following is an example of a simple function designed to return the full name of the current element. It is an example of a **RETURN** function with **NO ARGUMENTS**

```
define function !!nameCE() is STRING
  !ce = !!CE.flnn
  return !ce
endfunction
```

After a `PML REHASH ALL`, the function can be called by typing:

```
!name = !!nameCE()
```

After running, variable **!name** will be a string holding the full name of the current element.

If a function is defined with arguments, then these arguments will become local variables available within the function. These can then be used in expression, to control the function etc. The following is an example of a **RETURN** function with an **ARGUMENT**:

```
define function !!area(!radius is REAL) is REAL
  !circleArea = !radius.power(2) * 3.142
  return !circleArea
endfunction
```

After definition, the function can be used as part of an expression as it returns a real object. This means that common expressions/calculations can be written as a function and used as required.

```
!height = 64
!cylinderVolume = !!area(2.3) * !height
q var !CylinderVolume
```

4.3 PML Procedures

A function that does not return a value is typically described as a PML procedure. This means that as there is no return, the object-type does not need to be defined. The following example of a **NON-RETURN** function with **NO ARGUMENTS**:

```
define function !!setNameOfCE(!name is STRING)
  !!CE.name = !name
endfunction
```

The following is an example of a **NON-RETURN** function with **ARGUMENTS**:

```
define function !!setBoxPrimitiveSize(!x is REAL, !y is REAL, !z is REAL)
  if !!ce.type.eq(|BOX|) then
    !!CE.xlen = !x
    !!CE.ylen = !y
    !!CE.zlen = !z
  endif
endfunction
```

ⓘ *Notice how both the examples use the !!CE object (a global object which represents the current element). PML procedures are typically used to interact with defined global variables*

4.4 Recursive PML 2 Functions

PML functions may be called recursively. This can be used to produce iterative solvers.

```
define function !!fibonacci(!last is REAL, !previous is REAL)
  !next = !last + !previous
  !!resultArray.append(!next)
  if !!resultArray.Size().lt(100) then
    -- Here the function calls itself
    !!fibonacci(!next, !last)
  endif
endfunction
```

This iterative solver can be used to compile an array of Fibonacci numbers (an increasing series of numbers, where the next number is the sum of the previous two). The example requires a global variable !!resultArray before it will work.

4.5 Making Use of Methods on PML Objects

After a variable has been defined as a specific object-type, the methods of the object will be available on the variable. Making use of these methods can help manipulate the information within a function to ensure the correct outcome.

From the example on the previous page, **!radius** is defined as a **REAL** object. Therefore it has access to all the methods of a real object. The example makes use of the **.power()** method, and method which takes a real argument and raises the !variable to the power of the argument. The method returns the answer as a real object.

When working with built-in objects, refer to PDMS Software Customisation Reference Manual for the available methods and information about them

If you are working with different object-types, it is possible to switch been types. For example, there may be a need for a **REAL** to be seen as a **STRING** for use in an expression. The following would give an error:

```
!value = |56|
!result = !value * 2
```

As a **STRING** object cannot be multiplied by a **REAL**, an error will be returned. To avoid this error, the **.real()** methods can be used to return a **STRING** from a **REAL**. Note, the original variable remains a STRING, but it is seen as a REAL for the expression:

```
!value = |56|
!result = !value.real() * 2
```

There are equivalent methods available for the other standard objects, refer to PDMS Software Customisation Reference Manual.

4.5.1 Method Information

For each object-type in the Software Customisation Reference Manual, there is a table which lists the available methods and supporting information. For each method there will be:

NAME The name of the method or member. For example, a REAL object has a method named Cosine.

If there are any arguments, they are indicated in the brackets () after the name. For example, the REAL object has a method named BETWEEN which takes two REAL arguments.

RESULT The type of value returned by the method. For example, the result of the method Cosine is a REAL value. Some methods do not return a value: these are shown as NO RESULT.

PURPOSE This column tells you what the member or method does along with other information about the method or members.

*Note that for the system-defined PDMS object types, the members can be listed by using the **.attributes()** method and the methods can be listed using the **.methods()** method*

The following are some examples of **ARRAY** object methods to explain the different styles:

```
!numberOfNames = !nameStrings.size()
```

This method returns the number of elements currently in the array. This is an example of a **RESULT** method with **NO-EFFECT** on the original object.

```
!nameStrings.clear()
```

This method deletes the contents of the array, but not the array. This is an example of a **NO RESULT** method which does have an **EFFECT** on the original object.

```
!newNameArray = !nameStrings.removeFrom(5,10)
```

This method result removes 10 elements (starting at element 5) from the array. These elements are then returned by the method. This is an example of a RESULT method which does have an EFFECT on the original object. If you need to remove part of the array and do not need it, then it does not need assigning to a variable (e.g. just type `!nameStrings.removeFrom(5,10)`)

4.5.2 Method Concatenation

One of the advantages of object-orientated programming is that methods can be built-up within a single line of code. This means that complex manipulations can be made in fewer lines of PML. This process will only work if the previous method returns a suitable object for the following method. For example:

```
!line = 'hello world how are you'
!newline = !line.upcase().split().sort()
q var !line !newline
```

```
<STRING> 'hello world how are you'
<ARRAY>
      [1] <STRING> 'ARE'
      [2] <STRING> 'HELLO'
      [3] <STRING> 'HOW'
      [4] <STRING> 'WORLD'
      [5] <STRING> 'YOU'
```

In this example, the first two methods are valid for **STRING** objects. The **.split()** method returns an **ARRAY** object so the following method has to be a valid method for an **ARRAY** object.

4.6 Using the !!CE Object

!!CE is a special GLOBAL PML variable that always points to the current PDMS element. It is a **DBREF** object and its members are the attributes of the current element. Type `q var !!CE` onto the command line and compare it against the elements attributes (**Query>Attributes...**). You will notice the returned attribute information is the same of the members list of the !!CE object.

This means that the !!CE object can be used to assign the values of attributes to !variables. For example:

```
!branchHeadBore = !!CE.hbore
```

This assigns the **HBORE** attribute (taken from the current **BRAN** element) to the variable **!BranchHeadBore** making it real.

ⓘ *It will be necessary to check that HBORE is a valid attribute of the current element before running this line. It may cause an error.*

If the !!CE object member is an object itself, that object could also have members so further information be obtained. For example, obtain the east coordinate of a head of a **BRAN** element, either of the two following can be used:

```
!headPosition = !!CE.hpos
!headEasting = !headPosition.east
```

Or:

```
!headEasting = !!CE.hpos.east
```

If the !!CE object member is an object with built-in methods, then these methods can also be called:

`!PosWRTValve = !!CE.hpos.wrt(ZONE)` returns a POSITION object w.r.t the owning ZONE

This process can also be reversed and values can be applied to the attributes of the !!CE. This means that it is possible to record the current value of an attribute, modify and reassign back to the CE. For example, type out the following onto the command line:

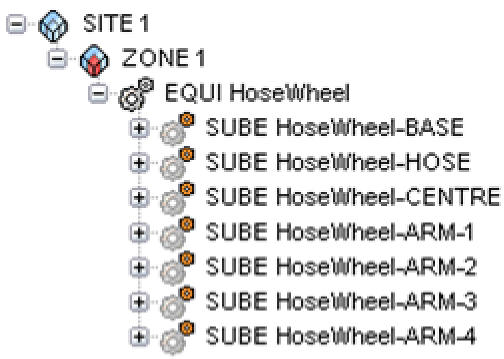
```
q POS
!position = !!CE.pos
!position.up = !position.up + 2000
!!CE.pos = !position
q POS
```

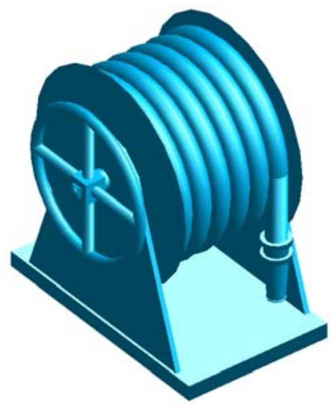
These lines will have moved the CE up by 2000. This same logic can be applied to other attributes, providing the object-type is considered.

Exercise 4 – Convert the HoseReel Macro into a PML Procedure

1

- You have been provided with **c1ex4.mac**, an old PML 1 style macro that builds a Hose Reel as a piece of equipment. The macro is old and has not been maintained. There are at least 5 problems with it.
- Debug the existing macro and convert it into a new PML procedure with **NO ARGUMENTS**





2

- Save the new file as **c1ex4.pmlfnc** under the PMLLIB searchpath.
- You may wish to create a new folder to keep the exercises separate from the examples
- Update the PMLLIB search path by typing `PML REHASH ALL`
- Test the function in the command window by typing `!!c1ex4()`

3

- Update the procedure and given it **TWO ARGUMENTS** which will represent the name of the EQUI and the diameter of the Hose Reel.
- Check the updated procedure to make sure the Hose Reel is created as expected. Consider error handling while you check the procedure.

An example of the completed procedure can be found in Appendix B

5 Collections

A very powerful feature of the PDMS database is the ability to collect and evaluate data according to rules. There are two available methods for collection that are valid in PML. The first is a command syntax PML 1 style which is explained in this chapter. The other method is by using a PML **COLLECTION** object and is not covered in this course.

More information on the PML COLLECTION object is given in the Programmable Macro Language Form Design Training Course

5.1 COLLECT Command Syntax (PML 1 Style)

The **COLLECT** syntax is based around three specific pieces of information:

- What element type is required?
- If specific elements are required, what is different about them?
- Which part of the hierarchy to look it?

If you wish to collect all the EQUI elements for the current ZONE, type the following:

```
var !equipment collect all EQUI for ZONE
q var !equipment
```

If you wish to collect all the piping components owned by a specific BRAN, type the following:

```
var !pipeComponents collect ALL with owner eq /200-B-4-B1 for SITE
q var !pipeComponents
```

if you wish to collect all the BOX primitives below the current element, type the following:

```
var !boxes collect all BOX for ce
q var !boxes
```

You do not need to specify level of the hierarchy to search within.

For more examples, refer to Chapter 11 of the Database Management Reference Manual and 2.3.10 Design Reference manual general commands

5.2 EVALUATE Command Syntax (PML 1 Style)

After elements have been collected through the **COLLECT** syntax, they are stored as an ARRAY. This array (like any array) can be processed using the **EVALUATE** syntax to provide further information

To get the names of all the elements held in **!equipment**, type the following:

```
var !equipmentNames evaluate NAME for ALL from !equipment
q var !equipmentNames
```

To get the fullnames of the elbows held within **!pipeComponents**, type the following:

```
var !elbows evaluate FLNN for all ELBO from !pipeComponents
q var !elbows
```

To get the names (without the leading slash) of the pumps in **!equipment**, type the following:

```
var !pumps evaluate NAMN for ALL with MATCHWILD(NAMN, |P*|) from !equipment
q var !pumps
```

To get the volume of all the boxes in **!boxes**, type the following:

```
var !volume eval (xlen * ylen * zlen) for ALL from !box
q var !volume
```

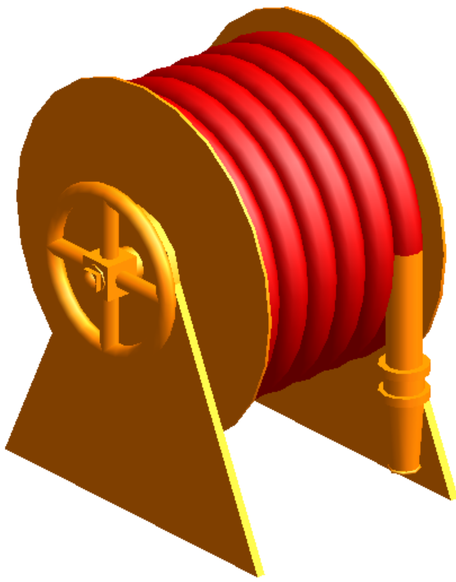
Notice how the expressions are command syntax and must return a BOOLEAN. Refer to the Software Customisation Reference Manual for more examples

Exercise 5 – Use Collection Syntax to Apply Colour to Hose Reel

1

- Write a function that applies colour to the hose elements and returns their names.
- PML 1 syntax should be used to collect the hose elements (shown in red below).
- The collection should be evaluated to find the element names.
- Colour can be applied to elements using the ENHANCE command.
- The function will require one Return, which should be an array element.

Details of the ENHANCE command can be found in the Design Reference Manual General Commands



2

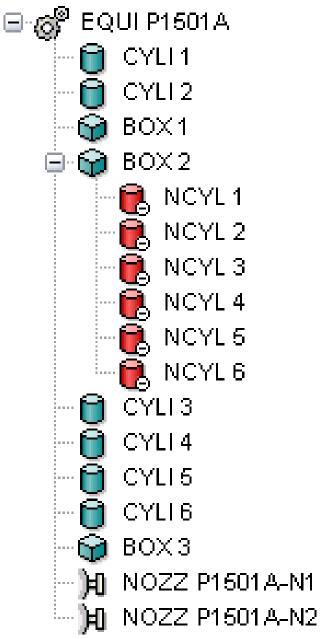
- Save the new file as **c1ex5.pmlfnc** under the PMLLIB searchpath.
- Update the PMLLIB search path by typing `PML REHASH ALL`
- Test the function in the command window by typing `!!c1ex5()`

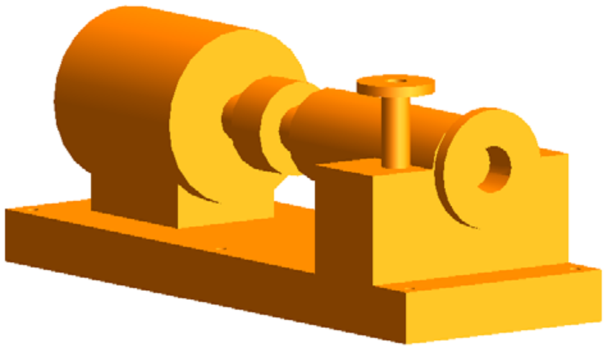
An example of the completed function can be found in Appendix B

Exercise 6 – Create a Function Based on a DB Listing

1

- Write a function that will build a piece of equipment
- Use a DB Listing as a basis for your Function
- Pick any piece of equipment and use the DB Listing utility to output the information (**Utilities>DB Listing…**)
- Convert the DB Listing macro into a Function





2

- Add some database marks at suitable points in the code to allow for Undo and Redo functionality.

3

- Save the new file as **c1ex6.pmlfnc** under the PMLLIB searchpath.
- Update the PMLLIB search path by typing `PML REHASH ALL`
- Test the function in the command window by typing `!!c1ex6()`

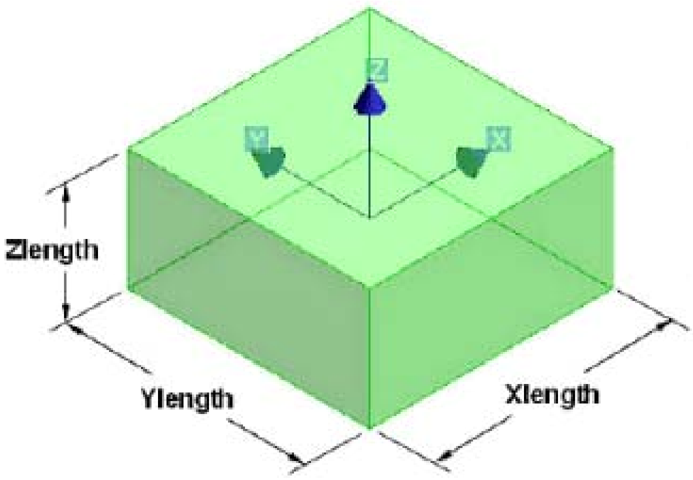
4

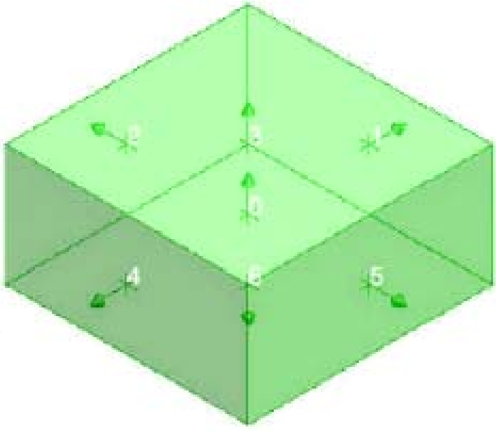
- Add arguments to the function to allow for customisation of the equipment, e.g. Name, Size of Base, etc.

An example of the completed function can be found in Appendix B

Appendix A – PDMS Primitives

Box (BOX)

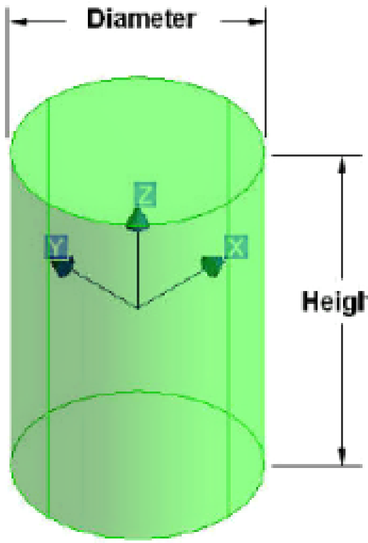


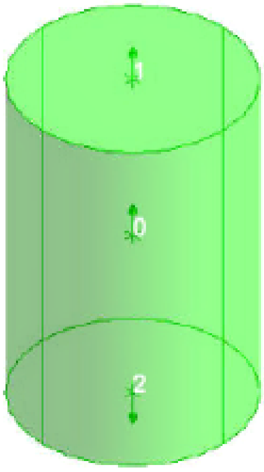


Specific geometric attributes:

Xlength	Length parallel to X axis
Ylength	Length parallel to Y axis
Zlength	Length parallel to Z axis

Cylinder (CYLI)

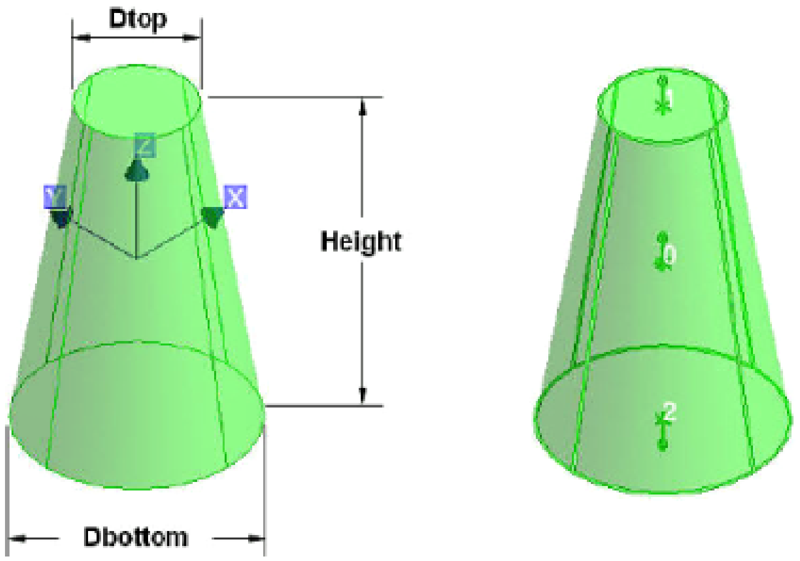




Specific geometric attributes:

Diameter	Diameter of cylinder
Height	Length parallel to Z axis

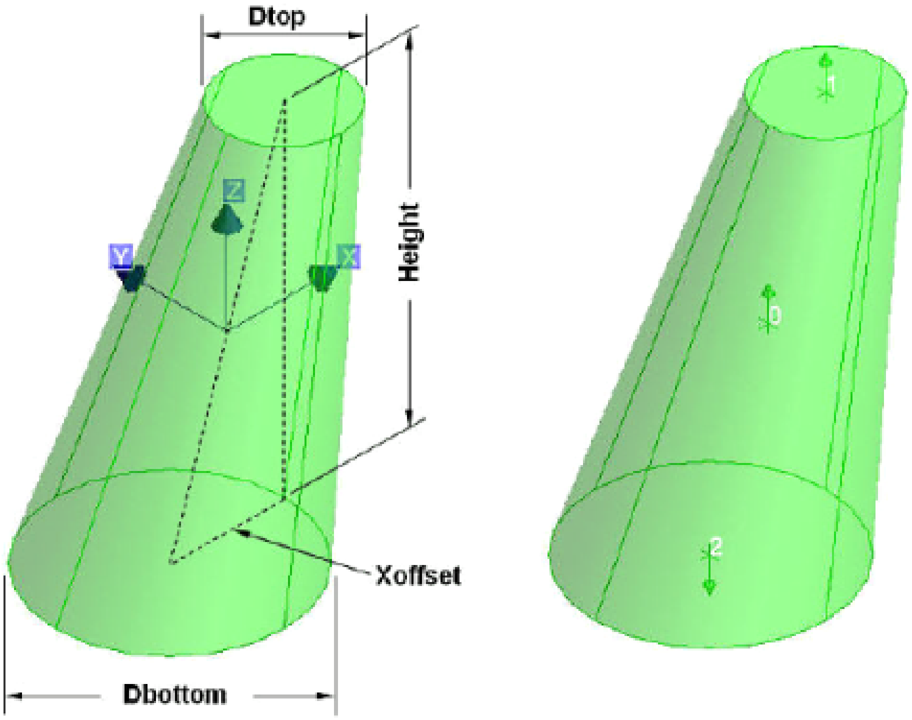
Cone (CONE)



Specific geometric attributes:

Dtop	Diameter at top of cone
Dbottom	Diameter at bottom of cone
Height	Length parallel to Z axis

Snout (SNOU)

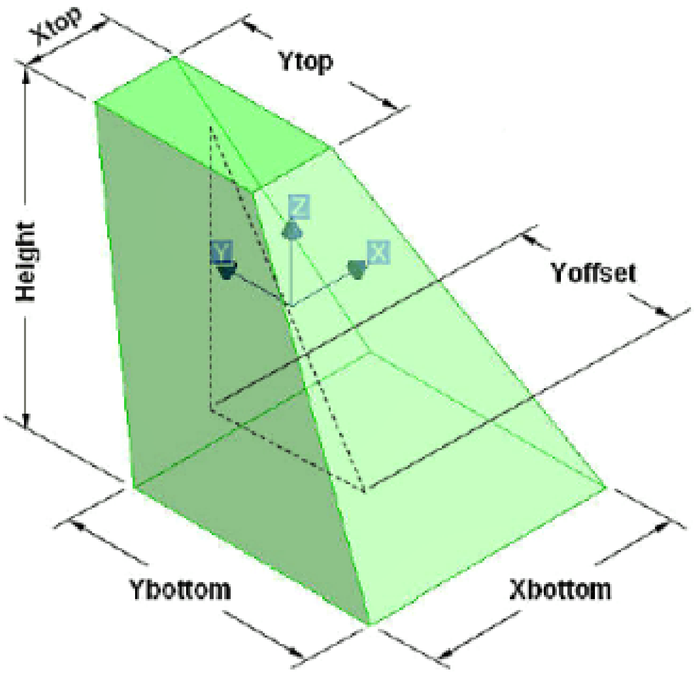


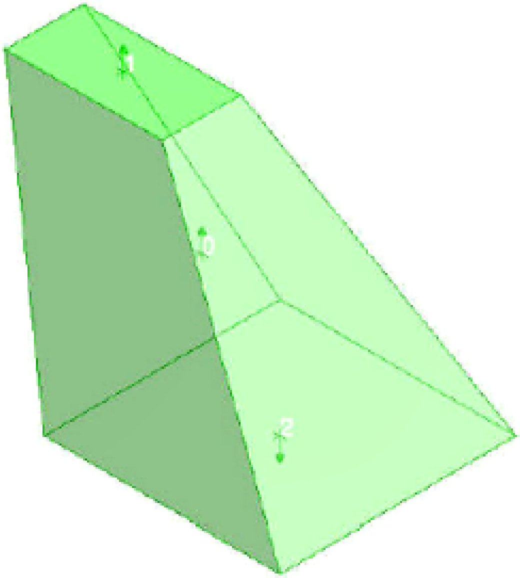
Specific geometric attributes:

Dtop	Diameter at top of snout
Dbottom	Diameter at bottom of snout
Xoffset	Offset of centre of top from centre of bottom on X axis
Yoffest	Offset of centre of top from centre of bottom on Y axis
Height	Length parallel to Z axis

ⓘ *Only an Xoffset is show in this example, however, both Yoffset and Xoffset may be set.*

Pyramid (PYRA)



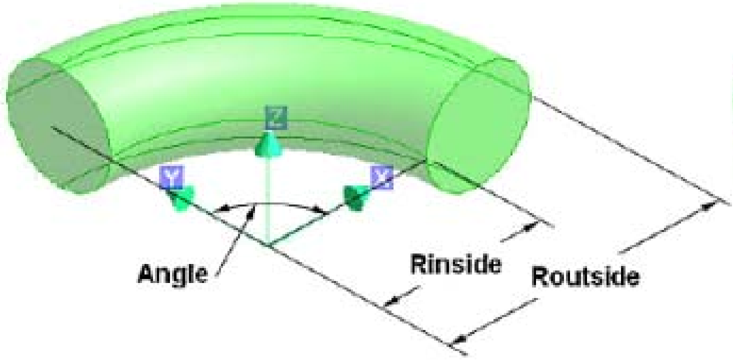


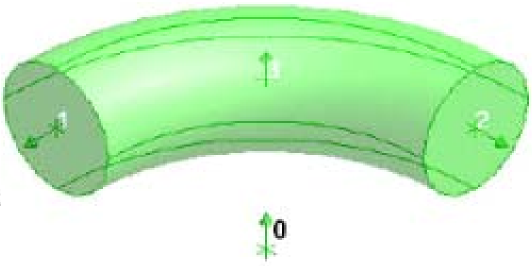
Specific geometric attributes:

Xbottom	Length of bottom of pyramid parallel to X axis
Ybottom	Length of bottom of pyramid parallel to Y axis
Xtop	Length of top of pyramid parallel to X axis
Ytop	Length of top of pyramid parallel to Y axis
Height	Length parallel to Z axis
Xoffset	Offset of centre of top from centre of bottom on X axis
Yoffset	Offset of centre of top from centre of bottom on Y axis

ⓘ *Only a Yoffset is show in this example, however, both Yoffset and Xoffset may be set.*

Circular Torus (CTOR)

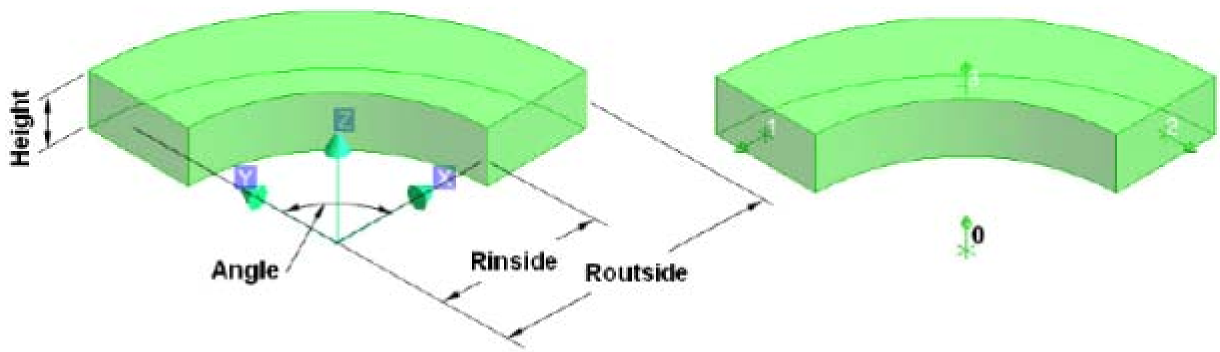




Specific geometric attributes:

Rinside	Inside radius in XY plane
Routside	Outside radius in XY plane
Angle	Subtended angle (maximum 180°)

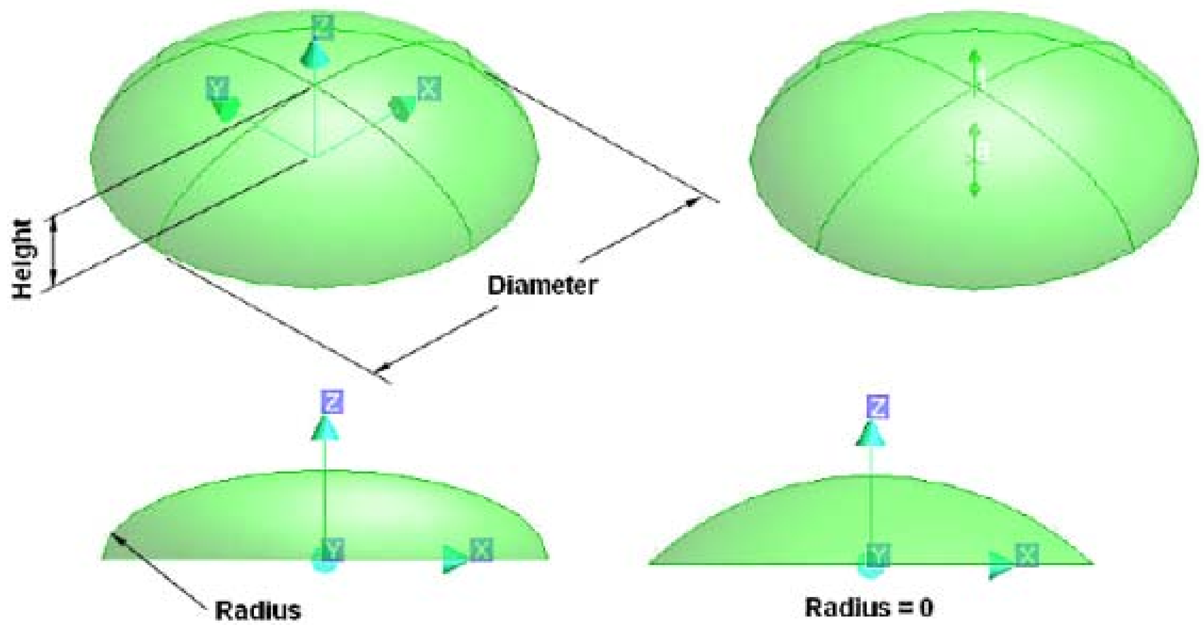
Rectangular Torus (RTOR)



Specific geometric attributes:

Rinside	Inside radius in XY plane
Routside	Outside radius in XY plane
Height	Length parallel to Z axis
Angle	Subtended angle (maximum 180°)

Dish (DISH)

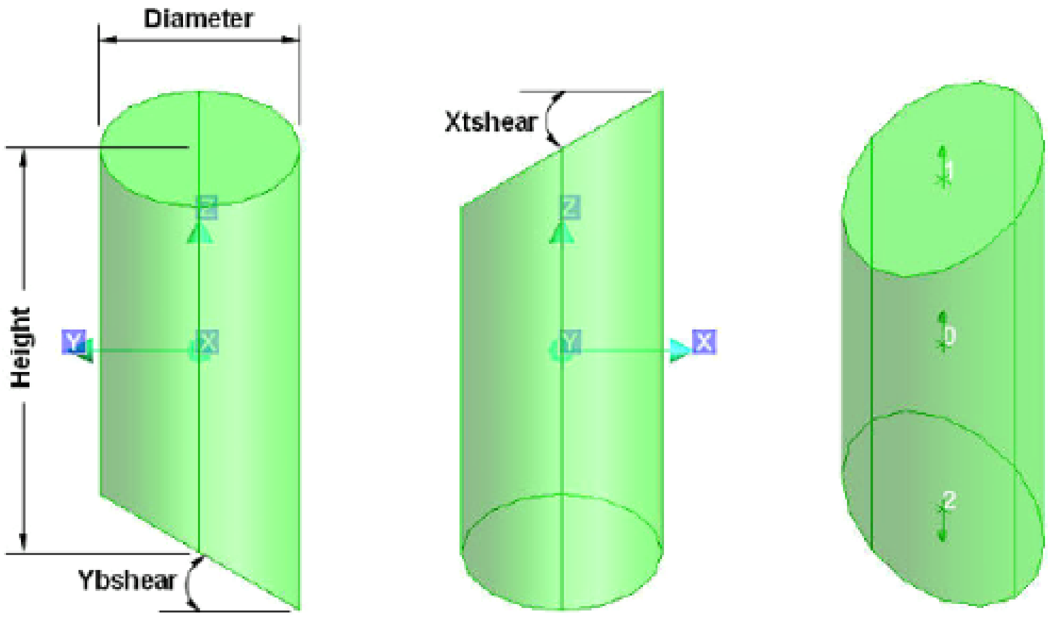


Specific geometric attributes:

Diameter	Diameter of dish in XY plane.
Height	Height of dish parallel to Z axis
Radius	Knuckle radius

ⓘ *If the knuckle radius is 0 then the dish is represented as a segment of a sphere. If the knuckle radius is greater than 0 then the dish is represented as a partial ellipsoid, generally used to represent a torispherical end to a vessel.*

Sloped Cylinder (SCYL)

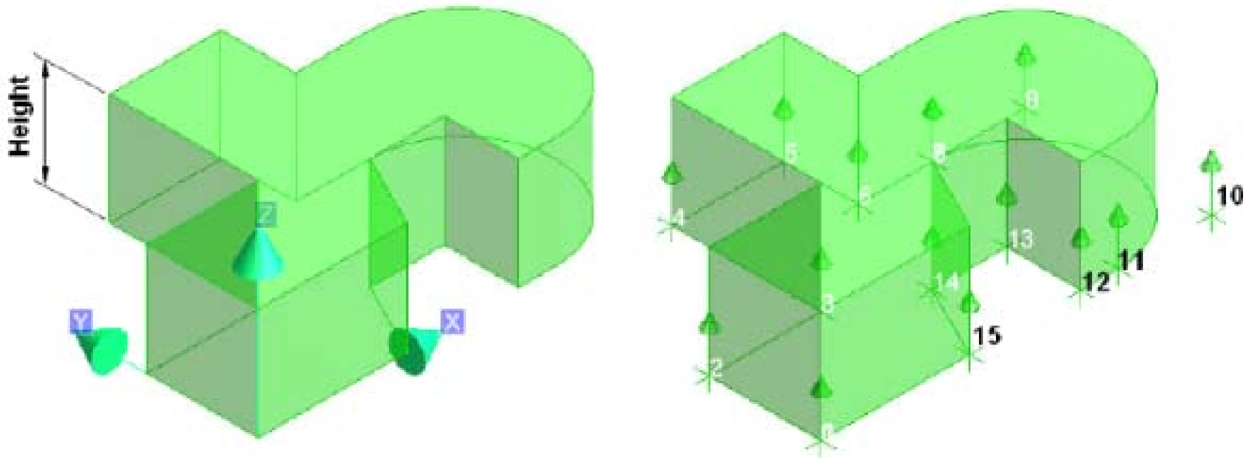


Specific geometric attributes:

Diameter	Diameter of sloped cylinder
Height	Length in Z axis from bottom centre to top centre
Xtshear	Inclination of top of cylinder in the XZ axis (in degrees)
Ytshear	Inclination of top of cylinder in the YZ axis (in degrees)
Xbshear	Inclination of bottom of cylinder in the XZ axis (in degrees)
Ybshear	Inclination of top of cylinder in the YZ axis (in degrees)

ⓘ *Only an Xtshear and Ybshear are shown in this example, however, Xtshear, Ytshear, Xbshear and Ybshear may be set in any combination to obtain the required results. The values for these attributes may be +ve or –ve.*

Extrusion (EXTR)

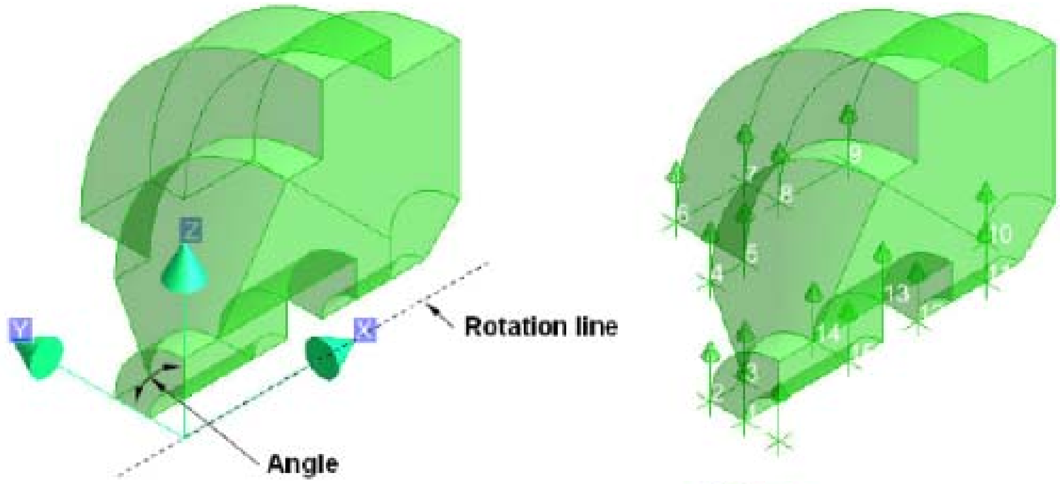


Specific geometric attributes:

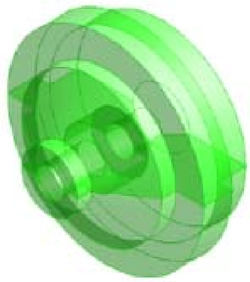
Height	Height of extrusion in Z axis

ⓘ *An extrusion is a 2D shape, defined by a series of vertices at each change in direction, extruded through a height. The primitive consists of three element types, i.e. EXTR, LOOP and VERTs.*

Solid of Revolution (REVO)



180° Rotation



360° Rotation

Specific geometric attributes:

Angle Rotation angle around X axis (selected rotation line)

ⓘ *A solid of revolution is a 2D shape, defined by a series of vertices at each change in direction, rotated through a specified angle around a specified rotation axis. The primitive consists of three element types, i.e. REVO, LOOP and VERTs.*

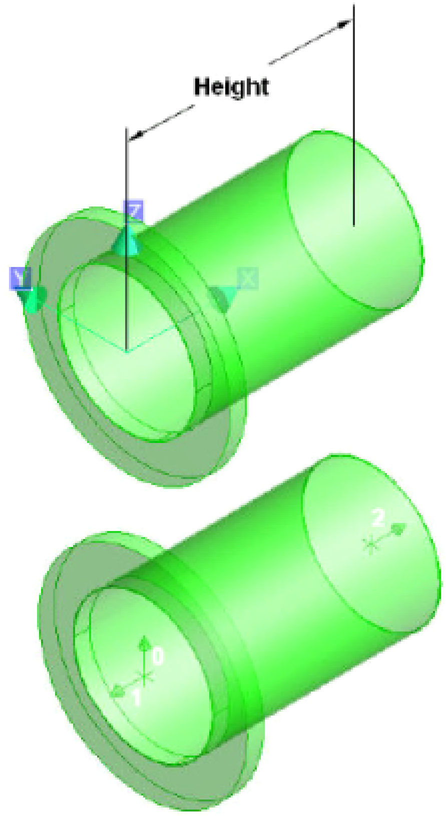
Nozzle (NOZZ)

Although a nozzle is classed as a primitive, it is unlike the other primitives in that its geometry is determined in Paragon as part of a catalogue component. Nozzles of different types and geometry may be constructed in Paragon to suit the requirements of the Piping Specification.

The specific nozzle type is referenced from Paragon using the **Spref** (Specification Reference) attribute.

Specific geometric attributes:

Height Height between nozzle face and end, i.e. from P1 to P2.



Appendix B – Example Code

This appendix contains examples of code that provide solutions to each of the exercises. The whole code has not been included in this appendix. Only new/modified code is included so it will require the user to read and understand this code.

Appendix B1 - Example c1ex2.mac

```
NEW EQUIPMENT /HandWheel

NEW BOX XLEN 100 YLEN 100 ZLEN 100
NEW CYLINDER DIAM 80 HEIG 5
CONN P1 TO P3 OF PREV
NEW BOX XLEN 50 YLEN 50 ZLEN 15
CONN P6 TO P2 OF PREV
NEW DISH DIAM 50 HEIG 15
CONN P2 TO P3 OF PREV

REM ALL
ADD /HandWheel AUTO /HandWheel
```

Appendix B2 - Example c1ex3.mac

```
$d1=HandWheel
$d2=500

NEW EQUIPMENT /$1

NEW SUBE /$1-Centre
NEW BOX /$1-Centre-Box XLEN 100 YLEN 100 ZLEN 100
NEW CYLINDER DIAM 80 HEIG 5
CONN P1 TO P3 OF PREV
  NEW BOX XLEN 50 YLEN 50 ZLEN 15
CONN P6 TO P2 OF PREV
  NEW DISH DIAM 50 HEIG 15
CONN P2 TO P3 OF PREV

NEW SUBE /$1-Arm-1
  NEW CYLINDER DIAM 50 HEIG ($2/ 2 - 75)
CONN P1 TO P2 OF /$1-Centre-Box
  NEW CTORUS RINS ($2 / 2 - 50) ROUT ($2 / 2)
CONN P0 TO P0 OF /$1-Centre-Box

do !p from 2 to 4
  NEW SUBE /$1-Arm-$!p COPY PREV
    ROTATE BY 90 ABOUT S
enddo

REM ALL
ADD /$1 AUTO /$1
```

Appendix B3 - Example c1ex4.mac (fixed)

```
$p
$p HOSE REEL MACRO (VERSION 1.0)
$p -------------------------
$p

$P ENTER REFNO OF THE HOSEREEL (e.g. HOSE-REEL-001)

VAR !NAME |HoseReel|
NEW EQUIP /$!NAME

$P
$P BUILDING HOSEREEL...

VAR !WHEELDIA (500)
VAR !REELDIA (1000)
```

Fix1: READ syntax replaced with standard variable declaration

```
NEW SUBE /$!NAME-BASE
NEW BOX XLEN 1200 YLEN 725 ZLEN 75

do !N FROM 0 TO 1
  NEW EXTRUSION HEIG 25
    POS W 0 N (-350 + ($!N * 675))  U 0
    ORI Y is E and Z is N
  NEW LOOP
    NEW VERTEX
      POS E 25 S 500 U 0
    NEW VERTEX
      POS E 25 N 500 U 0
    NEW VERTEX
      POS E ($!REELDIA / 2 + 250) N ($!WHEELDIA / 3) U 0
    NEW VERTEX
      POS E ($!REELDIA / 2 + 250 + $!WHEELDIA / 3) N ($!WHEELDIA / 3) U 0
      FRAD ($!WHEELDIA / 3)
    NEW VERTEX
      POS E ($!REELDIA / 2 + 250 + $!WHEELDIA / 3) S ($!WHEELDIA / 3) U 0
      FRAD ($!WHEELDIA / 3)
    NEW VERTEX
      POS E ($!REELDIA / 2 + 250) S ($!WHEELDIA / 3) U 0
enddo
NEW CYLINDER /$!NAME-BASE-AXLE DIAM 100 HEIG 750
  POS E 0 N 0 U ($!REELDIA / 2 + 250)
  ORI Y is E and Z is N
NEW NOZZLE
  POS E 0 N 450 U ($!REELDIA / 2 + 250)
  ORI Y is E and Z is U
  HEIG 100
  CATR /DICHTFL_C/DEZFBR0NN

NEW SUBE /$!NAME-HOSE
  POS E 0 N 0 U ($!REELDIA / 2 + 250)
  ORI Y is D and Z is S
  do !N FROM 0 to 1
    NEW CYLINDER DIAM $!REELDIA HEIG 10
      POS E 0 N 0 U (-305 + ($!N * 610))
  enddo
  do !I FROM 0 TO 5
    do !J FROM 0 TO 1
      NEW CTORUS RINS (($!REELDIA / 2) - 150) ROUT (($!REELDIA / 2) - 50) ANGL 180
        POS E 0 N 0 D (-250 + ($!I * 100))
        ROTATE BY ($!J * 180) ABOUT S WRT /*
    enddo
  enddo
  NEW CYLINDER DIAM 100 HEIG ($!REELDIA / 3)
    CONN P1 TO P2 OF PREV
  NEW CYLINDER DIAM 140 HEIG 25
    CONN P1 TO P2 OF PREV
  NEW CYLINDER DIAM 110 HEIG 50
    CONN P1 TO P2 OF PREV
  NEW CYLINDER DIAM 140 HEIG 25
    CONN P1 TO P2 OF PREV
  NEW CONE DTOP 110 DBOT 80 HEIG 200
    CONN P1 TO P2 OF PREV
  NEW REVOLUTION ANGL 360
    NEW LOOP
      NEW VERTEX
      NEW VERTEX
        POS W 10 N 0 U 0
      NEW VERTEX
        POS W 10 N 45 U 0
        FRAD 5
      NEW VERTEX
        POS W 0 N 45 U 0
        FRAD 5
  REVO
    CONN P1 TO P2 OF PREV
    ROTATE BY 90 ABOUT D

NEW SUBE /$!NAME-CENTRE
  NEW BOX /$!NAME-CENTRE-BOX XLEN 100 YLEN 100 ZLEN 100
    CONN P6 TO P2 OF /$!NAME-BASE-AXLE
  NEW CYLINDER DIAM 80 HEIG 5
    CONN P1 TO P3 OF PREV
```

Fix2: TO instead of TOO

Fix3: FRAD instead of PRAD

Fix4: S instead of S10E

Fix5: P1 to P2 instead of P1 to P1

```
  NEW BOX XLEN 50 YLEN 50 ZLEN 15
    CONN P6 TO P2 OF PREV
  NEW DISH DIAM 50 HEIG 15
    CONN P2 TO P3 OF PREV

NEW SUBE /$!NAME-ARM-1
  VAR !POS POS /$!NAME-CENTRE-BOX
  POS $!POS
  NEW CYLINDER DIAM 50 HEIG ($!WHEELDIA / 2 - 75)
    CONN P1 TO P2 OF /$!NAME-CENTRE-BOX
  NEW CTORUS RINS ($!WHEELDIA / 2 - 50) ROUT ($!WHEELDIA / 2)
    CONN P0 TO P0 OF /$!NAME-CENTRE-BOX

do !P FROM 2 TO 4
  NEW SUBE /$!NAME-ARM-$!P COPY PREV
    ROTATE BY 90 ABOUT U
enddo

REM ALL
ADD /$!NAME AUTO /$!NAME
EQUIP

$P
$P DONE!
$P
```

Fix6: BOX instead of BOXES

Appendix B4 - Example c1ex4.pmlfnc

```
define function !!c1ex4(!name is STRING, !wheelDia is REAL)
  NEW EQUIP /$!NAME

  VAR !REELDIA (1000)

  NEW SUBE /$!NAME-BASE
  NEW BOX XLEN 1200 YLEN 725 ZLEN 75

  do !N FROM 0 TO 1
  --------------------
  --
  --  As original Macro
  --
  --------------------
  EQUIP

endfunction
```

Middle part, same as middle part of original macro

Appendix B5 - Example c1ex5.pmlfnc

```
define function !!c1ex5() is ARRAY
  var !ctors collect all CTOR with rinside eq 350 for zone
  var !ctorNames evaluate FLNN for ALL from !ctors

  do !ctor values !ctors
    enhance $!ctor colour 2
  enddo

  return !ctorNames
endfunction
```

Appendix B6 - Example c1ex6.pmlfnc

```
define function !!c1ex6(!name is STRING)

  -------------------------------------------
  --
  --  As DB Listing but with changes to names
  --
  -------------------------------------------

  NEW EQUIPMENT /$!name
    POS E 0 N 0 U 0
    BUIL TRUE DSCO unset PTSP unset INSC unset
    NEW CYLINDER
```

!name variable used instead of original DB Listing name

```
    POS E 0 N 474 U 0 ORI Y is D and Z is N
    LEVE 0 4 DIAM 200 HEIG 800
  END
  NEW CYLINDER
    POS E 0 N 1074 U 0 ORI Y is D and Z is N
    LEVE 0 8 DIAM 370 HEIG 400
  END
  NEW BOX
    POS E 0 N 174 D 115
    LEVE 0 8 XLEN 510 YLEN 200 ZLEN 230
  END
  NEW BOX POS E 0 N 654 D 285
    LEVE 0 8 XLEN 510 YLEN 1390 ZLEN 110
    NEW NCYLINDER
      POS E 215 N 654 U 0
      LEVE 6 8 DIAM 25 HEIG 150
    END
    NEW NCYLINDER
      POS E 215 N 0 U 0
      LEVE 6 8 DIAM 25 HEIG 150
    END
    NEW NCYLINDER
      POS E 215 S 654 U 0
      LEVE 6 8 DIAM 25 HEIG 150
    END
    NEW NCYLINDER
      POS W 215 S 654 U 0
      LEVE 6 8 DIAM 25 HEIG 150
    END
    NEW NCYLINDER
      POS W 215 N 0 U 0
      LEVE 6 8 DIAM 25 HEIG 150
    END
    NEW NCYLINDER
      POS W 215 N 654 U 0
      LEVE 6 8 DIAM 25 HEIG 150
  END END
  NEW CYLINDER
    POS E 0 N 324 U 0 ORI Y is D and Z is N
    LEVE 5 8 OBST 0 DIAM 200 HEIG 500
  END
  NEW CYLINDER
    POS E 0 N 624 U 0 ORI Y is D and Z is N
    LEVE 5 8 OBST 0 DIAM 100 HEIG 100
  END
  NEW CYLINDER
    POS E 0 N 724 U 0 ORI Y is D and Z is N
    LEVE 5 8 OBST 0 DIAM 200 HEIG 100
  END
  NEW CYLINDER
    POS E 0 N 824 U 0 ORI Y is D and Z is N
    LEVE 5 8 OBST 0 DIAM 100 HEIG 100
  END
  NEW BOX
    POS E 0 N 1074 D 180
    LEVE 5 8 OBST 0 XLEN 250 YLEN 300 ZLEN 100
  END
  NEW NOZZLE /$!name-N1
    ORI Y is W and Z is U
    TEMP -100000 PRES 0 TPRESS 0 HEIG 155 DUTY unset
  END
  NEW NOZZLE /$!name-N2
    POS W 135 N 155 U 180 ORI Y is N and Z is E
    TEMP -100000 PRES 0 TPRESS 0 HEIG 180 DUTY unset
END END

OLD /$!name-N1
  CREF BRANCH /100-B-8-B2  TAIL
  CATR SCOMPONENT /AAZFBD0NN
OLD /$!name-N2
  CREF BRANCH /50-B-9-B1  HEAD
  CATR SCOMPONENT /AAZFBD0JJ

REM ALL
ADD /$!name AUTO /$!name
EQUIP
endfunction
```

!name variable used instead of original DB Listing name

!name variable used instead of original DB Listing name